

CI112 Arquitetura de Computadores - 2025/1 - UFPR

Especificação do Trabalho 3

Em duplas, os alunos deverão desenvolver os projetos de implementação da **arquitetura Sagui**, versões Vetorial e VLIW. Trata-se de uma arquitetura de 8-bits, do tipo load-store endereçada por byte, que foi desenvolvida especialmente para essa disciplina. A lista de instruções bem como seus formatos são dados nas próximas páginas.

Adicionalmente, as implementações deverão utilizar os opcodes reservados para instruções novas a sua escolha.

O trabalho deve ser desenvolvido em duplas, com cada dupla entregando dois projetos (um Vetorial, e um VLIW) e um relatório. Recomenda-se a seguinte ordem para o desenvolvimento do trabalho:

1. Desenvolvimento com diagrama de caixas do projeto do caminho de dados (datapath) dos processadores.
2. Desenvolvimento com diagrama de caixas do projeto interno das ULAs, pensando no código das ULAs a serem usados para cada operação.
3. Desenvolver tabelas com sinais de controle.
4. Desenvolvimento no Logisim Evolution dos projetos com componentes e fios.
5. **Escreva um programa de teste para cada processador:**

- Uma soma de 2 vetores teste de **12 posições** fazendo $R=A+B$. Seu código deve inicializar os vetores A, B e R. Os vetores devem estar alinhados na memória.

Você deve utilizar pelo menos um loop para implementar a soma ou para inicializar o vetor (você deve mostrar em algum ponto que sabe trabalhar com laços).

R deverá iniciar com zero.

$A = \{0,1,2,3,4,5,6,7,8,9,10,11\}$.

$B = \{20,21,22,23,24,25,26,27,28,29,30,31\}$.

No caso do processador vetorial os dados estarão distribuídos nas memórias dos PEs.

6. O assembly do código testado deve estar presente no relatório para conferência do professor.
7. **Configure seu logisim para ele ser mais rigoroso. Para funcionar corretamente, adicione um botão de reset ao teu projeto, para zerar os registradores PC e RegisterBank.**

Project -> Options -> Memory elements start in unknown state

Project -> Options -> Gate output when undefined -> Error for undefined inputs

Regras Gerais de Entrega e Apresentação

A implementação será feita no logisim evolution, onde poderão ser utilizados todos os componentes pré-prontos ali existentes. Cada dupla deverá entregar os diagramas dos processadores bem como os projetos no logisim evolution. A entrega será feita pelo Moodle dividida em duas partes

- Relatório em PDF contendo os **diagramas dos processadores**, os **projetos das suas respectivas ULAs**, as **tabelas de sinais de controle** com eventuais detalhes do projeto.

- Projetos no formato **Logisim Evolution**

As datas limite de entrega serão sempre às 6h (a.m.) do dia, impreterivelmente. (não confundir com 18h)

Casos não tratados no enunciado deverão ser discutidos com o professor.

A cópia do trabalho (plágio), acarretará em nota igual a Zero para todos os envolvidos.

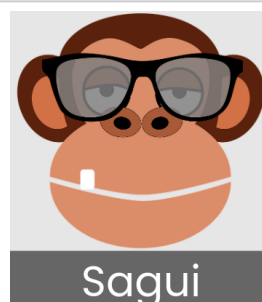
Os trabalhos deverão ser apresentados de forma oral pelos alunos. A nota irá considerar domínio do tema, robustez da solução e rigorosidade da metodologia.

Sagui em Bando (Vetorial)

Vector Architecture					
Opco de	Tip o	Menem onico	Nome	Operação	
Scalar - SPE					
0000	R	s.ld	Load	SR[ra] = M[SR[rb]]	
0001	R	s.st	Store	M[SR[rb]] = SR[ra]	
0010	I	s.movh	Move High	SR[1] = {Imm., SR[1](3:0)}	
0011	I	s.movl	Move Low	SR[1] = {SR[1](7:4), Imm.}	
0100	R	s.add	Add	SR[ra] = SR[ra] + SR[rb]	
0101	R	s.sub	Sub	SR[ra] = SR[ra] - SR[rb]	
0110	R	s.and	And	SR[ra] = SR[ra] & SR[rb]	
0111	R	s.brzr	Branch On Zero Register	if (SR[ra] == 0) PC = SR[rb]	
Vector - VPE					
1000	R	v.ld	Load	VR[ra] = M[VR[rb]]	
1001	R	v.st	Store	M[VR[rb]] = VR[ra]	
1010	I	v.movh	Move High	VR[1] = {Imm., VR[1](3:0)}	
1011	I	v.movl	Move Low	VR[1] = {VR[1](7:4), Imm.}	
1100	R	v.add	Add	VR[ra] = VR[ra] + VR[rb]	
1101	R	v.sub	Sub	VR[ra] = VR[ra] - VR[rb]	
1110			Reservado - instruções adicionais		
1111			Reservado - instruções adicionais		
			4x Vector PE	Scalar PE	
SR -> Scalar register			4 Regs por PE - Sendo o 1º é o ID {0,1,2,3} do PE ele é hardwired (não muda) - Os outros 3x são de propósito geral (GP)	4 Regs - Sendo o 1º igual a ZERO hardwired (não muda) - Outros 3x de propósito geral (GP)	
VR -> Vectorial register			VR0 = {0,1,2,3} dependendo do PE	SR0 = 0	
			1 Memória por PE Considere que o vetor estará distribuído entre as memórias	1 Memória exclusiva	
Em um dado ciclo apenas 1 dos PEs atuam, ou VPE ou SPE. Não existem instruções que atuem de forma escalar e vetorial ao mesmo tempo. Enquanto um atua o outro recebe NOP.					

Tipo R							
7	6	5	4	3	2	1	0
opcode				Ra		Rb	

Tipo I							
7	6	5	4	3	2	1	0
opcode				Imm			



Sagui de Rabo Longo (VLIW)

VLIW Architecture				
Opcode	Tip o	Menemoni co	Nome	Operação
Controle				
0000	R	brzr	Branch On Zero Register	if (R[ra] == 0) PC = R[rb]
0001	I	brzi	Branch On Zero Immediate	if (R[0] == 0) PC = PC + Imm.
0010	R	jr	Jump Register	PC = R[rb]
0011			Reservado - instruções adicionais	
Dados				
0100	R	ld	Load	R[ra] = M[R[rb]]
0101	R	st	Store	M[R[rb]] = R[ra]
0110	I	movh	Move High	R[0] = {Imm., R[0](3:0)}
0111	I	movl	Move Low	R[0] = {R[0](7:4), Imm.}
Aritmética				
1000	R	add	Add	R[ra] = R[ra] + R[rb]
1001	R	sub	Sub	R[ra] = R[ra] - R[rb]
Lógica				
1010			Reservado - instruções adicionais	
1011	R	or	Or	R[ra] = R[ra] R[rb]
1100	R	not	Not	R[ra] = ! R[rb]
1101	R	slr	Shift Left Register	R[ra] = R[ra] << R[rb]
1110	R	srr	Shift Right Register	R[ra] = R[ra] >> R[rb]
NOP = Free Slot				
1111	R	nop	No Operation	
			VLIW	4 Lanes Fixos
			4 Lanes = 8*4 bits	1º (Bits mais significativos) LD / ST / MOV
			4 Regs GP (General Purpose)	2º BR / JUMP
				3º ULA
				4º (Bits menos significativos) ULA

- A memória pode ser endereçada em 4 bytes para facilitar a obtenção dos dados.
- Podemos ter 1 controle por FU a fim de reduzir a complexidade do controle.
- Consideramos que não haverão instruções dependentes na mesma palavra, e que o compilador não permitirá duas ou mais operações de escrita ao mesmo registrador na mesma instrução.

Tipo LD							
7	6	5	4	3	2	1	0
opcode				Ra		Rb	

Tipo BR							
7	6	5	4	3	2	1	0
opcode				Imm			

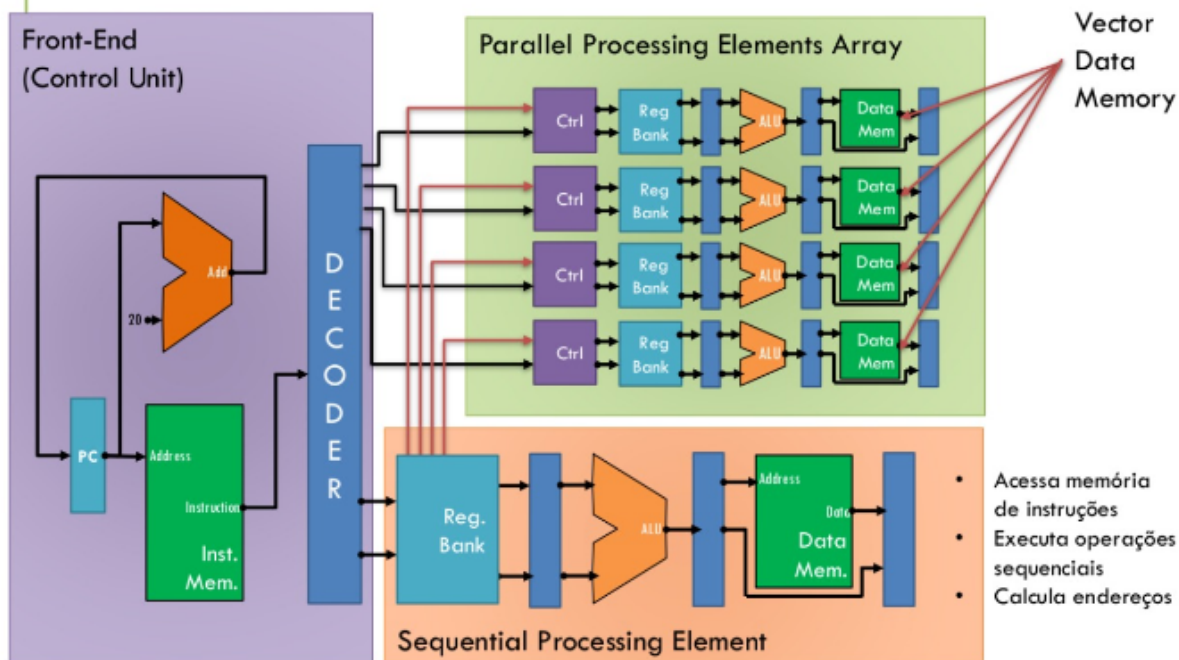
Tipo ULA							
7	6	5	4	3	2	1	0
opcode				Ra		Rb	



- Detalhes específicos:

As figuras abaixo mostram datapaths de inspiração para o projeto do Sagui:

Vetorial



VLIW

